

Basics of System Engineering Field Manual

Memorize the whole deck the easy way: one story, the exact exam points, and a memory hook on every topic.

Module **MMC-01a** Lecturer **Prof. Dr. M. Górka** Source **1.pptx** · 154 slides

THD · NuW · Mechatronic & CPS



Meet Robi & Cy — your memory anchors

You're an engineering team building Robi, a cleaning robot — your first Cyber-Physical System. Robi has a **body you can touch** (sensors, motors, wires = *physical*) and a **brain that thinks & talks to the internet** (software, AI = *cyber*). Watching over your shoulder is Cy, a wise owl, who keeps hooting “What is a system? What is design?”

Every slide just answers one question: “What is Robi made of, how complex is it, and how do clever engineers design and build a whole SYSTEM instead of bolting parts together?”

Keep Robi + Cy in your head and the chapter tells itself.

★ **MASTER KEY** → **System = WHAT (purpose)** · **Design = HOW** · then **ENGINEER the whole system: Requirements → Ideation → Design → Test → Release (ringed by Risk & Change)**

See the skeleton before the bones

Basics of System Engineering is step 1 of System Engineering:

1 BASICS ★ → 2 Requirements → 3 Risk → 4 Change → 5 Ideation → (6 Design) → (7 Test) → (8 Project Setup)

The chapter has two big halves — and they are literally the definition of **System Design**. First we zoom *into* a system (what it's made of, how complex it is); then we zoom *out* to how you **engineer** a whole one.



System = WHAT (the purpose) · Design = HOW. Hold that split and every topic has a home.

The Memory Movie

Don't memorize sentences — your brain hates random letters. Instead **close your eyes and play this movie** of “*Building Robi, from idea to finished robot*” in slide order. Each picture is a concept, so when you replay the movie the exam answer just appears. Read it 3 times, then try to replay it with your eyes shut.



SCENE 1

Cy the owl lands on the workbench and hoots “What is a system?” → **a group of parts working for ONE purpose, fenced off from the world.**



SCENE 2

Cy points outside: **galaxy, leaf, lung** grew with no human (NATURAL); on the bench a **piano, plane, robot** were built (ARTIFICIAL). → **examples of systems.**



SCENE 3

A glass tank shows a **coral reef** (life + machine pumps). → **CLASSIFICATION: natural / artificial / biotechnical; Open · Partly-open · Isolated doors.**



SCENE 4

Robi's brain runs a **loop**: “where I should be” vs “where I am”, fix the gap. → **CYBERNETICS — the control loop; “cyber” = the brainy half.**



SCENE 5

A giant **wheel** spins: a tiny DNA-datasheet core, a ring of properties, 3 pie-slices (mech/E-E/SW). → **the SYSTEM WHEEL.**



SCENE 6

Robi is a **conveyor belt**: Sensor → wire → Brain → wire → Motor. → **SYSTEM STRUCTURE (SIPOC).**

SCENE 7



Russian nesting dolls: Supersystem $\supset$ System $\supset$ Sub-system $\supset$ Component. → **SYSTEM LEVELS / tiers.**



SCENE 8

You tip Robi out into a **pile of bricks** (hardware, software, liquids, even people & ideas). → **COMPONENTS.**



SCENE 9

One sulphur crumb is colourless, but a **pile glows yellow**. → **EMERGENCE** ("1+1=3").



SCENE 10

A **storm cloud** (VUCA) rolls in; a neat **subway map** = merely complicated, the **weather** = truly complex. → **COMPLEXITY.**



SCENE 11

Robi's parts are tied into a **spider web** — count the knots. → **DEGREE OF CONNECTIVITY** (clustering coefficient).



SCENE 12

You **simulate** Robi 1000× on screen; a **butterfly** flaps and it explodes. → **SYSTEM DYNAMICS + CHAOS THEORY.**



SCENE 13

An artist with a **paintbrush** turns requirements into drawings, models, code. → **DESIGN.**



SCENE 14

A **factory** with 3 lanes (core/management/support) and BMW's Idea→Offer→Order→Delivery. → **SYSTEM DESIGN PROCESS.**



SCENE 15

A **big gear-loop**: Requirements → Ideas → Design → Test → Release, ringed by Risk & Change . → **SYSTEM ENGINEERING (the steps).**

SCENE 16



A runner at the start line sweats about globalization, interfaces, variants, safety. → **the CHALLENGES.**



SCENE 17

A digital ghost-twin floats beside the real Robi; code pops out of a diagram by itself.
→ **MBSE / MDSD / Digital Engineering.**



Replay in one breath: owl asks → galaxy vs piano → reef tank → brain-loop → spinning wheel → conveyor belt → nesting dolls → brick pile → glowing sulphur → storm cloud → spider web → simulation + butterfly → paintbrush → factory → gear-loop → sweating runner → digital twin.



Picture-hooks for the tricky lists

Same trick for every list that wouldn't stick — turn it into one strong, weird image:



Natural systems

Galaxy (life), Leaf (photosynthesis), Lung (breathing). **Grew by themselves, no human.**



Artificial systems

Piano (sound), Plane (fly), Robot (expressions). **Built by people.**



Open • Partly • Isolated

Three doors on Robi: **wide-open** • a mail-slot (OBD car) • welded shut.



Mechatronic vs CPS

A lone washing machine (standalone) vs a smart home where the washer texts the solar panels (**CPS / IoT**).



System Wheel

Bike wheel: hub = **DNA datasheet**, spokes = properties, 3-colour rim = mechanics / E-E / software.



SIPOC structure

“See It Process Output Customer” = **Sensor** → **Connector** → **Logic** → **Connector** → **Actuator**.



System Levels

Nesting dolls. **System + Component always exist**; Super/Sub optional. Your level depends on who holds the doll (tier).



Emergence

One sulphur crumb is colourless, the **pile** glows; one molecule has no pressure, the gas does. **Ignorance $\neq$ emergence**.



VUCA storm

A Very Ugly Chaotic Atmosphere: **V**olatile, **U**ncertain, **C**omplex, **A**mbiguous.



Complicated vs Complex

Subway map (rules known $\rightarrow$ waterfall) vs weather (rules unknown $\rightarrow$ **SCRUM**).



Multi-Agent System

An ant colony solving a maze with no boss; an ant that claims a job **broadcasts** so others don't repeat it.



Connectivity

A spider web: count nodes + connections; a **two-way strand counts as 2** (clustering coefficient).



Chaos theory

A butterfly over turbulent gas / the internet: tiny flap, **huge unpredictable effect**.



SE steps

Requirements $\rightarrow$ Ideation $\rightarrow$ Design $\rightarrow$ Test $\rightarrow$ Release, with **Risk** ☢️ & **Change** 🔄 hugging the loop. ISO 15288.



Accountability

After Robi-car crashes, a crowd of suspects lines up. **Nobody knows who's to blame** — the future challenge.



Digital twin / MDSD

A ghost-Robi mirrors the real one live (Digital Eng.); a vending machine spits code from a diagram (**MDSD / Simulink**).



How to actually lock it in: (1) read a scene, (2) **shut your eyes and re-see the picture**, (3) say the real meaning out loud. Do the 17-scene movie + picture-hooks twice today and once tomorrow — then test yourself with the **flashcards**. Seeing beats reading; testing beats re-reading.

Taught like a story — with the points you must write

01 System Design = two halves

STORY

Before you touch a screwdriver, Cy splits the word. **System Design** = “**the System**” (its purpose) + “**the Design**” (how you build it). Robi's *purpose* is “clean the floor”; the *design* is the wiring diagram and code that make it happen.

WRITE IN EXAM

- **System Design consists of two parts:** the **System** (purpose) and the **Design** (how).

 **System = WHAT · Design = HOW.** Two words, two halves of the whole course.

02 What *is* a system?

STORY

Cy hoots, “What is a system?” Look at Robi: lots of **different parts** all chasing **one purpose**, and you can draw a **fence** separating it from the world. If the parts were all the same with no shared purpose, it's just a **pile** (accumulation), not a system.

WRITE IN EXAM — DEFINITION

- **A system = a group of associated components that work together for the same purpose and that can be bordered from the environment.**
- Components have **different characteristics** — otherwise it's an **accumulation**, not a system.
- Systems are **clearly bordered**; sometimes **partial overlaps at interfaces**.
- **Natural** (no human): Galaxy (life) · Leaf (photosynthesis) · Lung (breathing).
- **Artificial** (man-made): Piano (sound) · Airplane (fly) · Robot (expressions); also search engine, sports team, organization.



Different parts · one purpose · a **fence** around it. No fence or all-same-parts = not a system.

03 System Classification

STORY

Cy shows a **coral-reef tank** — living coral + machine pumps. Systems sort into **3 bins**, man-made ones into **3 more**, and each is tagged by **how open** its doors are.

WRITE IN EXAM

- **Types of systems:** Natural · Artificial · **Biotechnical**.
- **Man-made types:** **Technical** (machines, tools) · **Social** (family, groups, work) · **Sociotechnical** (technical + social influencing each other; e.g. information/work system).
- **By exchange with environment:** **Open** (full interfaces — info/material/energy) · **Partly open** (small pre-defined data, e.g. **OBD** in a car) · **Isolated** (nothing exchanged).
- Parts can differ: a robot's **tracking** = **isolated**, its **map** → **open Bluetooth**.



3 doors on Robi → wide-Open · mail-slot (Partly, “OBD”) · welded-shut (Isolated).

04 Cyber, Cybernetics & CPS

STORY

Why the “Cyber” in Cyber-Physical? **Cyber** = **short for cybernetics** = **controlling machines using feedback** — like Robi's brain asking “am I where I should be?” and correcting. Cy draws the **control loop**: Target – Current = a gap → Controller → Actuator → the world reacts → Sensor reads it → loop again.

WRITE IN EXAM

- **Cyber = short form of cybernetics. Cybernetics = control of machines** incl. feedback of **sense organs** (analogy to lifeforms) and **social interactions** (communication).
- **Control loop (in order):** Target value → difference (target – current) → Controller → Command → Actuator → Action → System/Environment → Reaction → Sensor → Current value → fed back (–).
- **“Cyber” umbrella term:** Cybersecurity, Cyberspace, Cyberwarfare, Cybercrime, Cyber-attack, **Cyborgs** (organic + biomechatronic crossover).

 **Cyber = feedback brain.** The loop is a circle: **gap → fix → measure → repeat.**

04a Mechatronic vs CPS

STORY

A lone washing machine is **mechatronic** — software + hardware, sealed, standalone. Now let the washer text the solar panels to run when the sun shines — that networked smart-home is a CPS.

WRITE IN EXAM

- **Mechatronic system** = software + hardware (mechanics & electronics); **standalone/closed**. E.g. washing machine.
- **CPS = intercommunicating** software (esp. embedded, via internet = cyber) + hardware (= physical); for **large-scale, distributed networks (IoT)**. E.g. home automation linking washer + photovoltaic.
- **No sharp deviation** between mechatronic and CPS.
- Applied to this lecture: **physical** (sensors, actuators, PCB & wires) + **cyber** (processing unit, AI, general software) → a CPS.

 **Mechatronic = lonely washer. CPS = washer that texts the solar panels.**

05 System Wheel

STORY

Cy spins a **bicycle wheel** — the course's master diagram. The **hub** is Robi's "DNA" datasheet (performance, weight, cost...). Spokes are **extra properties** (safety, EMC, thermal...). The **rim is painted in 3 colours** = the building trades: **mechanics**, E/E, **software**. The **customer** stands at the edge. (Full detail in Requirement Management.)

WRITE IN EXAM

- Rings (inside→out): **core "DNA" properties (datasheet)** → **extended properties** (acoustics, efficiency, data/security, design-for-production/service, safety, EMC, sustainability, reliability, thermal, volumetric) → **realization domain** (mechanics / E/E / software) → **customer**.
- Customer raises requirements **on system level**.



Wheel = Hub (DNA) · Spokes (properties) · 3-colour Rim (mech/E-E/SW) · Customer at the edge.

06 System Structure (SIPOC)

STORY

Every system, says Cy, is secretly a **conveyor belt**: something comes *in*, gets *processed*, goes *out*. That's **SIPOC**. For Robi: a **Sensor** feels → a **wire** carries → the **Brain/Logic** decides → a **wire** carries → a **Motor** acts.

WRITE IN EXAM

- Every process = **SIPOC** (Supplier · Input · Process · Output · Customer).
- High-level structure: **Sensor** → **Connector** → **Logic** → **Connector** → **Actuator/Visualizer**.
- **Connector** = PCB / wire / wireless · **Logic** = software / IC / state machine / AI · **Actuator/Visualizer** = motor / valve / HMI / database.
- **ChatGPT**: prompt page → server → AI magic → browser → output field.
- **Sorting machine**: colour sensor 0–5 V → TXT controller → threshold picks pusher 1/2/3 → motor driver → pusher fires.



“See It Process Output Customer” = Sensor → Connector → Logic → Connector → Actuator. A conveyor belt, always.

07 System Levels & Tiers

STORY

Pull Robi apart like **Russian nesting dolls**: the big **Supersystem** holds **Systems**, which hold **Sub-systems**, which hold **Components**. A higher level is just the **sum** of the dolls inside.

WRITE IN EXAM

- Levels (top→bottom): **Supersystem (4, rare) → System (3) → Sub-system (2) → Component (1)**.
- A higher level = the **sum of the elements below**; lower-level count is **variable but always > 1**.
- **Supersystem & Sub-system optional; System & Component always exist**.
- **Same element, different level** by viewpoint: a car = system for the car-maker, but the **screw** = system for the screw supplier.
- In industry, supplier levels = “**tiers**” (manufacturer → tier-1 → tier-2 → tier-3 → ...).



Nesting dolls. System + Component are the two that **MUST** exist; Super/Sub are optional. Your level depends on who holds the doll (tier).

08 Components

STORY

Tip Robi out onto the bench — a **pile of bricks**. Some you can touch (**hardware, software, liquids**), some you can't (**talents, roles, ideas**). Each brick is **fenced off** and does its **own job**.

WRITE IN EXAM

- **Components = the basic elements of a system.**
- **Real (material):** Hardware (electronics/mechanics/hydraulics) · Software (functions/interfaces) · Material/consumables (chemicals/liquids).
- **Immaterial:** Talents & Roles · Ideas.
- Components are **clearly bordered** and have **different functions/tasks**.



Brick pile — touchable bricks (HW/SW/liquids) + invisible bricks (people/ideas).

09 Emergence

STORY

Drop **one** sulphur crumb — colourless. Pour a **pile** — it glows **yellow**. The colour belongs to the *group*, not the crumb. That surprise extra property = **emergence**, “ $1+1=3$ ”. But if something only *seems* magic because we're **ignorant**, that's NOT emergence — it vanishes once science catches up.

WRITE IN EXAM

- **Emergence = a system has other properties than the simple sum of its components (“ $1+1=3$ ”).**
- Examples: **single sulphur atom = no colour, many = yellow; single molecule = no pressure, gas = pressure.**
- **Ignorance is NOT emergence** — effects merely unknown (low state-of-the-art) that disappear as knowledge evolves.



Glowing sulphur pile. The property lives in the crowd, not the crumb. Don't-know $\neq$ emergence.

10 Complexity & VUCA

STORY

A **storm cloud** rolls over the workshop — the modern world is **VUCA**: a *Very Ugly Chaotic Atmosphere*. Then Cy plays a game: a **subway map** vs the **weather** — which is complex? The map is only **complicated** (hard but all rules known); the weather is truly **complex** (rules unknown, all tangled).

WRITE IN EXAM

- **VUCA**: **V**olatile (discontinuity, high speed, big swings) · **U**ncertain (unclear outcomes, low predictability) · **C**omplex (many layers, fuzzy boundaries) · **A**mbiguous (no simple description, multiple interpretations).
- Complexity is set by the **amount, order & interaction** of components; **complex systems can't be broken into easy problems**.
- **Properties of complex systems**: Agent-based · Non-linear · Emergence · Open · Self-organised · Self-regulating · Path-dependencies (history matters).
- **CAS** = changes itself (stock exchange, brain). **MAS/ABM** = self-organised agents, no boss, broadcast jobs (ant colony, DAI).
- **Measure complexity (IT)**: Computational Complexity Theory via resource demand (time, storage, chip size, # parallel chips).



VUCA = the storm. Complicated = **subway map** (waterfall). Complex = **weather** (SCRUM).

11 Degree of Connectivity

STORY

Tie Robi's parts together with **string** until they form a **spider web**. The more knots and threads, the higher the **connectivity**. Rule: parts in the **same unit** may talk; parts in **different units on the same level must NOT** — if they do, you drew the **border wrong** (→ wrong responsibilities!).

WRITE IN EXAM

- Same-level, **same-unit** elements **may** interact & affect each other.
- **No** interaction between same-level elements in **different units** — else the **border is wrong** → **risk of wrong responsibilities**.
- Interactions swap **information / material / energy**, are **interdependent**, **not unidirectional**.
- **Degree of connectivity** = (amount of different **nodes**) + (amount of causal **connections**).
- Measured by the **clustering coefficient** (exact, no rounding; a **bidirectional** link = 2).



Spider web — count knots + threads; a two-way thread = 2. Cross-unit threads = a wiring mistake.

12 System Dynamics & Chaos Theory

STORY

You can't sketch Robi's full behaviour on a napkin — too tangled. So you **simulate** the whole system on a computer and run it **1000 times**. That's **System Dynamics**. Then a **butterfly** flaps, the simulation goes wild and unpredictable — **Chaos Theory**.

WRITE IN EXAM

- **System Dynamics** = model & simulate the whole system (complex systems can't be described easily/linearly). Variants: qualitative & quantitative.
- **Qualitative** — find **feedback loops**: **reinforcing** (positive coupling) & **balancing** (negative coupling); easier; good overview.
- **Quantitative** — model & simulate; **real numbers**; many runs; flow charts/process views; finds **non-linear effects**.
- **Chaos Theory** = non-linear dynamic systems; **"butterfly effect"** — small change → big effect, prediction impossible (turbulent gas, internet).



Simulate 1000× = System Dynamics (**Qualitative** = arrows/loops, **Quantitative** = numbers). **Butterfly** = **Chaos**.

13 Design

STORY

Now an **artist with a paintbrush** turns Robi's requirements into something real — drawings, models, code. But the artist can't ignore the **factory**: design, integration and production all **pull on each other**. A design nobody can manufacture (a ball inside a ball, or all-gold wires) is useless.

WRITE IN EXAM

- **Design = translating requirements (respecting the system purpose) into a (technical) solution.**
- **Deliverables:** Plan · Drawings/schematics · Models · Software code · ...
- **Design, integration & production can't be strictly divided** — they influence each other; build **prototypes** and **test** upfront.
- Without production know-how → risk of a **non-producible** (ball-in-ball) or **non-commercial** (all-gold wires) part.



Paintbrush turns requirements → drawings/models/code; but the **factory is always watching**.

14 System Design Process

STORY

Zoom out to the whole **company factory**. Every process is one of **3 lanes**: the **core** lane (money made with customers/suppliers), the **management** lane (steering), the **support** lane (helpers). BMW's core lane reads like a relay: **Idea → Offer → Order → Delivery → Customer-Care**.

WRITE IN EXAM

- **3 process classes:** **Core** (business with customers & suppliers; **value-adding**) · **Management** · **Support**. Flow: **customer requirement → core process → customer satisfaction**.
- **BMW core process:** **ItO** (Idea→Offer) · **OtO** (Offer→Order) · **OtD** (Order→Delivery) · **DtCC** (Delivery→Customer-Care).
- **ItO** = development from requirement → ideas/concepts → implementation; purchasing procures; after test & validation + industrialisation → release for production. Aim: **optimal product, all requirements met**.

🗨️ **3 lanes = Core · Management · Support.** BMW relay: **I-O-O-D** → Idea, Offer, Order, Delivery (then Care).

15 System Engineering — the 7 steps ★

🗨️ STORY

Here's the **big gear-loop** that actually builds Robi. Five steps run in order — **Requirements** → **Ideation** → **Design** → **Test** → **Release** — and two more, **Risk** ☢️ and **Change** 🔄, hug the whole loop the entire time. Defined by a standard, **ISO 15288**.

📖 WRITE IN EXAM — SE IS

- an **interdisciplinary** field of engineering + engineering management;
- **design, integrate & manage complex systems over their life cycle**; defined in **ISO/IEC 15288**.

📖 THE STEPS (IN ORDER)

- **1. Requirement Management** — analyse problem & requests (Customer, Law, LeLe); break into requirements; define system requirements.
- **2. Ideation / Concept** — creative search for ideas (maybe research!); define functions; create **system architecture**; break (super)system → subsystem → component; break requirements into levels; **interface design**.
- **3. Design / Realization** — model, simulate, analyse; implement into **HW/E-E/SW**; integrate levels; document. → **core engineering work**.
- **4. Test & Verification & Validation** — test design, the “**test of the test**”, planning, execution (virtual + HW). → **proof by measurement** every requirement is met.
- **5. Release** — sign off; release to production; take over **accountability** to the customer.
- **Risk Management** (surrounding) — check risks to product/time/cost/safety (e.g. FMEA); deviation management.
- **Change Management** (surrounding) — update on changing requirements (new law, customer change of mind, LeLe/field issues); **version control**.

🗨️ **Gear-loop: R-I-D-T-R** (Requirements, Ideation, Design, Test, Release) wrapped by **Risk** ☢️ + **Change** 🔄. Standard = **15288**. Without it → the “tree-swing” cartoon.

16 System Engineering – Challenges

STORY

A **runner at the start line** is sweating because modern CPS projects are brutal: the world went global, everything connects to everything, customers want endless variants, and you can't even prove the thing is safe.

WRITE IN EXAM

- **Globalization** → many suppliers; rising **legal** specs; **unclear responsibilities** (fuzzy boundaries).
- **Interdisciplinary** work mandatory; **diverse interfaces** → **consistent control of all interfaces**.
- **Complex requirements** → more dynamics; **more variants** → component kits (reusable units). (VW Golf ≈ 425 billion wiring-harness combos.)
- **Reduced cycle time** → hard **quality-cost-time** trade-off (VUCA).
- **Distributed sensor network** → **sensor fusion** (camera + lidar). **Communication handling** (when? which path?). **Information gathering** (lots of changing data).
- **Hard to verify** (can't stop it, who's root cause, hard to debug, AI changes). **Safety** (trust network data; new attack points).



Sweating runner. Everything got bigger, more connected, more varied, harder to verify & secure. “Who crashed the robot-car?” = nobody can tell → the future challenge.

17 MBSE, MDSD & Digital Engineering

STORY

Finally, three power-tools. **MBSE** = the System Wheel **with a working model** so every discipline reads the same picture. **MDSD** = a **vending machine** that spits out finished code/tests/docs straight from a diagram (Simulink → C). **Digital Engineering** = a **ghost-twin** of Robi you can watch and poke live.

WRITE IN EXAM

- **MBSE (Model-Based Systems Engineering)** = System Wheel + a **model**; cross-discipline communication; all info **independent of realization domains**; needs a **harmonized modelling language**. Languages: **UML & SysML**.
- **MBSE benefits**: visualization · architectures/dependencies → **traceability** · automated tests · **different views** · foreseeable change impact · direct code generation (MDSD).
- **MDSD / MDD** = auto-generate **functional software, tests, documentation** from the model **without coding** (Simulink → C). Pro: abstraction, speed, fitting test cases. Con: more initial effort, tricky manual-code integration, harder troubleshooting.
- **Digital Engineering** = successor of MBSE (INCOSE / GfSE); uses **“digital twins”** (live + virtual interaction); uses MBSE · AI · CAD/CAE · additive manufacturing · VR/AR.

 **MBSE = shared model · MDSD = vending machine (diagram → code) · Digital Engineering = ghost-twin.** Languages = UML/SysML; councils = INCOSE/GfSE.

18 Fields of Application + Summary

WRITE IN EXAM

- **Fields**: Traffic control · Production/logistics · **Medical devices** (smartwatch emergency call) · Energy/smart grid · **Automotive** (lane keeping, auto-stop, distance control).
- **Summary**: rising **complexity** (components + **VUCA** + interconnections) forces engineers to develop **whole systems & functions**, not single sub-components; **system engineering** handles this; **holistic thinking** avoids failures and reaches the **overall optimum**, not a singular one.

 Complexity went up → **engineer the WHOLE system** → a holistic optimum beats a local one.

YOUR PROGRESS

0 / 18 topics learned

Flashcards

Read the question, say the answer *out loud*, then flip. Cover the deck and run it twice — that's how it sticks.

All flashcards (Q/A)

1. Define a "system".

A group of associated components that work together for the same purpose and that can be bordered from the environment.

2. System vs accumulation?

A system's parts have different characteristics and one shared purpose; a mere pile of same parts is an accumulation.

3. Two parts of System Design?

The System (the purpose / 'what') and the Design (the 'how').

4. Natural systems — 3 examples?

Galaxy (life), Leaf (photosynthesis), Lung (breathing) — developed without human input.

5. Artificial systems — examples?

Piano (sound), Airplane (fly), Robot (expressions); also search engine, sports team, organization.

6. 3 types of systems?

Natural, Artificial, Biotechnical.

7. 3 types of man-made systems?

Technical (machines/tools), Social (family/groups/work), Sociotechnical (technical + social influencing each other).

8. Classify by exchange with environment?

Open (full interfaces), Partly open (small pre-defined data, e.g. OBD), Isolated (nothing exchanged).

9. What does 'Cyber' stand for?

Short form of 'cybernetics'.

10. Define cybernetics.

Control of machines including feedback of sense organs (lifeform analogy) and feedback of social interactions (communication).

11. Cybernetic control loop in order?

Target → difference (target – current) → Controller → Command → Actuator → Action → System/Environment → Reaction → Sensor → Current value → fed back.

12. What is a cyborg?

An organic ('man') + biomechatronic ('machine') crossover.

13. Define a mechatronic system.

A merge of software + hardware (mechanics & electronics); can be standalone/closed. E.g. washing machine.

14. Define a Cyber-Physical System (CPS).

Intercommunicating software (esp. embedded, via internet = cyber) + hardware (= physical); for large, distributed networks (IoT). E.g. home automation.

15. Mechatronic vs CPS — sharp distinction?

No — there is no sharp deviation between them.

16. Applied to this lecture, a system =?

Physical (sensors, actuators, PCB & wires) + cyber (processing unit, AI, software) → a CPS.

17. System Wheel inner ring (core)?

The 'DNA' of the system — core properties of a one-page datasheet (performance, cost, weight, reliability, size...).

18. 3 realization domains in the System Wheel?

Mechanics, E/E (electrics/electronics), Software.

19. What is SIPOC?

Supplier – Input – Process – Output – Customer; a general way to describe any process.

20. High-level system structure (SIPOC applied)?

Sensor → Connector → Logic → Connector → Actuator/Visualizer.

21. What is a 'Connector' in the structure?

The link between blocks — PCB, wire, wireless, etc.

22. What counts as 'Logic'?

Software, IC, calculator, state machine, AI, etc.

23. Map ChatGPT onto the structure.

Prompt page (sensor) → to server (connector) → AI magic (logic) → to browser (connector) → output field (visualizer).

24. 4 system levels (top→bottom)?

Supersystem (rare) → System → Sub-system → Component.

25. Which levels always exist?

System and Component; Supersystem and Sub-system are optional.

26. Why must a lower level have >1 element?

Else it would equal the higher level, so that level wouldn't exist.

27. Same element, different level — example?

A car is a 'system' for the car-maker but a 'supersystem'; a screw is a 'system' for the screw supplier.

28. What are 'tiers'?

Supplier levels in industry: manufacturer → tier-1 → tier-2 → tier-3, etc.

29. What is a component?

The basic element of a system; clearly bordered, with different functions/tasks.

30. Kinds of components (real & immaterial)?

Real: hardware, software, material/consumables. Immaterial: talents & roles, ideas.

31. Define emergence.

A system has other properties than the simple sum of its components ('1+1=3').

32. Two examples of emergence?

One sulphur atom = no colour but many = yellow; one molecule = no pressure but a gas has pressure.

33. Why is ignorance NOT emergence?

Some effects are merely unknown (low state-of-the-art) and disappear as knowledge evolves.

34. What does VUCA stand for?

Volatile, Uncertain, Complex, Ambiguous.

35. VUCA — Volatile?

Discontinuity; high speed & intensity; large ranges of fluctuation.

36. VUCA — Uncertain?

Unclear outcomes; unknown future / low predictability; things seem to emerge from nothing.

37. VUCA — Complex?

Elements connected in many layers; many systems; clear boundaries hard to define.

38. VUCA — Ambiguous?

No simple description; situations interpreted differently.

39. Properties of complex systems?

Agent-based, non-linear, emergence, open, self-organised, self-regulating, path-dependencies.

40. Complicated vs Complex?

Complicated = all known, predictable, removable by training → waterfall. Complex = (partly) unknown, not predictable, can't be removed → agile/SCRUM.

41. Subway map: complex or complicated?

Complicated — many lines but clear rules; all elements known; much effort but predictable.

42. Weather: complex or complicated?

Complex — not all elements known; all interconnected; the kind of interconnection unknown.

43. What is a Complex Adaptive System (CAS)?

A system that changes itself (adapts/learns), not static. E.g. stock exchange, human brain.

44. What is a Multi-Agent System (MAS)?

Several agents that communicate; autonomous, distributed, self-organised (no boss); broadcast jobs to avoid double work. E.g. ant colony, DAI.

45. How is complexity measured in IT?

Computational Complexity Theory via resource demand: computing time, storage, chip size, number of parallel chips.

46. Rule: same-level elements in different units?

Must NOT interact — if they do, the element border is wrong (risk of wrong responsibilities).

47. What defines the degree of connectivity?

The amount of (different) nodes and the amount of causal connections between them.

48. How is connectivity measured?

With the clustering coefficient (exact; a bidirectional connection counts as 2).

49. What is System Dynamics?

Modelling & simulating overall systems to analyse complex/dynamic systems. Variants: qualitative & quantitative.

50. Qualitative system dynamics?

Identifies feedback loops: reinforcing (positive) & balancing (negative). Easier; good overview.

51. Quantitative system dynamics?

Models & simulates the whole system; real numbers; many runs; flow charts; finds non-linear effects.

52. What is Chaos Theory?

Field of non-linear dynamic systems; 'butterfly effect' — small change → big effect, prediction impossible (turbulent gas, internet).

53. Define 'design' (system design).

The process of translating requirements (respecting the system purpose) into a (technical) solution.

54. Deliverables of a design?

Plan, drawings/schematics, models, software code, etc.

55. Why can't design, integration & production be split?

They influence each other; ignoring production capability risks a non-producible (ball-in-ball) or non-commercial (all-gold wires) part.

56. 3 classes of company processes?

Core, Management, Support.

57. What is a core process?

Value-adding business with customers & suppliers: customer requirement → core process → customer satisfaction.

58. BMW's simplified core process phases?

ItO (Idea→Offer), OtO (Offer→Order), OtD (Order→Delivery), DtCC (Delivery→Customer Care).

59. What is 'ItO'?

Idea to Offer: development from requirement → ideas/concepts → implementation; after test, validation & industrialisation → release.

60. Define System Engineering.

Interdisciplinary engineering + management; design, integrate & manage complex systems over their life cycle; defined in ISO/IEC 15288.

61. Steps of the System Engineering process?

Requirement Mgmt → Ideation/Concept → Design/Realization → Test & Verification & Validation → Release; surrounded by Risk & Change Management.

62. Requirement Management does what?

Analyses the problem & requests (Customer, Law, LeLe); breaks it into requirements; defines the system requirements.

63. Ideation / Concept does what?

Creative search for ideas; define functions; create system architecture; break system→subsystem→component; break requirements into levels; interface design.

64. Design / Realization does what?

Model, simulate, analyse; implement into HW/E-E/SW; integrate levels; document. (Core engineering work.)

65. Test & Verification & Validation does what?

Test design, 'test of the test', planning, execution (virtual + HW). Proof by measurement that every requirement is met.

66. Release does what?

Sign off; release to production; take over accountability (per requirements & law) to the customer.

67. What does the 'tree-swing' cartoon show?

The result of building a system WITHOUT system design & requirement management — every department builds something different.

68. Name SE challenges.

Globalization, many suppliers, legal specs, unclear responsibilities, interdisciplinary work, diverse interfaces, complex requirements, many variants, reduced cycle time, sensor fusion, communication, verification, safety.

69. Why is sensor fusion needed?

To describe an object, data from different sensors distributed by location must be combined (e.g. camera + lidar).

70. Wiring-harness variants in a typical car?

About 425 billion combinations (e.g. VW Golf), due to customization.

71. What is MBSE?

Model-Based Systems Engineering — System Wheel + a model; cross-discipline communication; all info independent of realization domains; needs a harmonized modelling language.

72. Benefits of MBSE?

Visualization, architectures/dependencies → traceability, automated tests, different views, foreseeable change impacts, direct code generation.

73. MBSE modelling languages?

UML (Unified Modeling Language) and SysML (Systems Modeling Language).

74. What is MDSD (MDD)?

Model-Driven Software Development — auto-generate functional software, tests & docs from the model without coding (e.g. Simulink → C).

75. MDSD advantages & disadvantages?

Pro: higher abstraction, faster, fitting test cases. Con: more initial effort, tricky manual-code integration, harder troubleshooting.

76. What is Digital Engineering?

Successor of MBSE (INCOSE / GfSE); uses digital twins to see performance live & interact virtually; uses MBSE, AI, CAD/CAE, AM, VR/AR.

77. Example fields of application for CPS?

Traffic control, production/logistics, medical devices, energy/smart grid, automotive (lane keeping, auto-stop, distance control).

78. Core summary message of the chapter?

Rising complexity forces developing whole systems/functions, not single parts; system engineering handles this; holistic thinking reaches the overall optimum.

3 Practice Questions + model answers

Try to answer on paper before clicking **Reveal**. The model answers show the exact points an examiner rewards.

Q1 Define a “system” and distinguish a mechatronic system from a cyber-physical system. Give one example of each.

TYPE: DEFINITIONS · ~8 MARKS

REVEAL MODEL ANSWER

✓ MODEL ANSWER

A system is a group of associated components that work together for the same purpose and that can be bordered from the environment. The components have different characteristics — otherwise it would be a mere **accumulation**. Borders are clear, though interfaces may partly overlap.

- **Mechatronic system:** a merge of **software** + **hardware** (mechanics & electronics); can be a **standalone, closed** system. Example: a **washing machine**.
- **Cyber-Physical System (CPS):** a merge of **intercommunicating software** (esp. embedded, via the internet = *cyber*) + **hardware** (= *physical*); suited to **large-scale, distributed networks** (IoT). Example: **home automation** linking the washing machine with the photovoltaic system.

There is **no sharp deviation** between the two. Applied to this lecture a system = physical (sensors, actuators, PCB & wires) + cyber (processing unit, AI, software).

Q2

Explain the difference between a “complicated” and a “complex” system, define VUCA, and name what makes a system complex (its properties).

TYPE: STRUCTURED / THEORY · ~10 MARKS

REVEAL MODEL ANSWER

✓ MODEL ANSWER

Complicated systems have **all elements, rules and dependencies known**; they need much effort but are **predictable** and can be mastered by training/knowledge → managed with a **waterfall model** (e.g. a subway grid plan). **Complex** systems have (partly) **unknown elements, rules and dependencies**; they are **not predictable** and cannot be “removed” → managed with **agile / SCRUM** (e.g. the weather).

VUCA = **Volatile** (discontinuity, big fluctuations), **Uncertain** (unclear outcomes, low predictability), **Complex** (many layers, fuzzy boundaries), **Ambiguous** (no simple description).

Properties of complex systems: agent-based, non-linear, emergence, open, self-organised, self-regulating, and path-dependencies (behaviour depends on history, not just the current state).

Q3

You are building Robi, a cleaning robot. (a) Draw/describe its high-level system structure (SIPOC). (b) Name the steps of the System Engineering process. (c) Give two challenges of engineering such a CPS.

TYPE: APPLICATION · ~9 MARKS

REVEAL MODEL ANSWER

✓ MODEL ANSWER

(a) Every system follows the SIPOC chain: **Sensor** → **Connector** → **Logic** → **Connector** → **Actuator/Visualizer**. For Robi: a dirt/obstacle **sensor** → a **wire** (connector) → the **logic** (software/AI on a processing unit) → a **wire** → the drive **motor/brush** (actuator).

(b) System Engineering (ISO/IEC 15288) runs: **Requirement Management** → **Ideation/Concept** → **Design/Realization** → **Test & Verification & Validation** → **Release**, with **Risk Management** and **Change Management** surrounding the whole loop.

(c) Any two of: increasing **interdisciplinary** work & diverse **interfaces** (consistent control needed); many **product variants**; **distributed sensor network** needing **sensor fusion**; **hard to verify** (can't stop the whole system, AI keeps changing); **safety** (untrusted network data, new attack points); unclear **accountability** (e.g. who is liable when an autonomous car crashes).

— LAST 5 MINUTES BEFORE THE EXAM

One-screen cheat-card

Two halves	System (purpose) + Design (how)	WHAT + HOW
System def	parts · one purpose · bordered	different parts + a fence
Natural vs Artificial	grew vs built	Galaxy/Leaf/Lung vs Piano/Plane/Robot
Open/Partly/Isolated	full · OBD · sealed	3 doors on Robi
Cybernetics	control via feedback loop	gap→fix→measure→repeat
Mechatronic vs CPS	standalone vs networked/IoT	lone washer vs washer-texts-solar
System Wheel	DNA core · properties · 3 domains · customer	bike wheel, 3-colour rim
SIPOC structure	Sensor→Conn→Logic→Conn→Actuator	“See It Process Output Customer”
Levels	Super·System·Sub·Component	nesting dolls (System+Comp always)
Components	HW/SW/liquids + talents/ideas	brick pile (touchable + invisible)
Emergence	1+1=3, group property	glowing sulphur pile
VUCA	Volatile·Uncertain·Complex·Ambiguous	the storm cloud
Complicated vs Complex	known→waterfall / unknown→SCRUM	subway map vs weather
MAS	self-organised agents, no boss	ant colony (broadcast jobs)
Connectivity	nodes + connections, bidirectional=2	spider web
System Dynamics	model & simulate (qual loops / quant numbers)	simulate 1000×
Chaos	non-linear, butterfly effect	butterfly over turbulent gas
Design	requirements → technical solution	paintbrush + watching factory
Process classes	Core · Management · Support	3 lanes; BMW I-O-O-D

SE 7 steps	Req·Ideation·Design·Test·Release + Risk·Change	gear-loop, ISO 15288
Challenges	global · interfaces · variants · verify · safety	sweating runner; “who crashed the car?”
MBSE/MDSD/DigEng	model / code-vending / digital twin	share·vend·ghost; UML/SysML; INCOSE

Basics of System Engineering · CPS MMC-01a · study build · keep Robi & Cy in your head 🤖 🛡️

Downloads: [PDF](#) · [Anki flashcards](#) · [memorize guide](#) · [notes](#)